Aula 30: Estado e Props no React

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

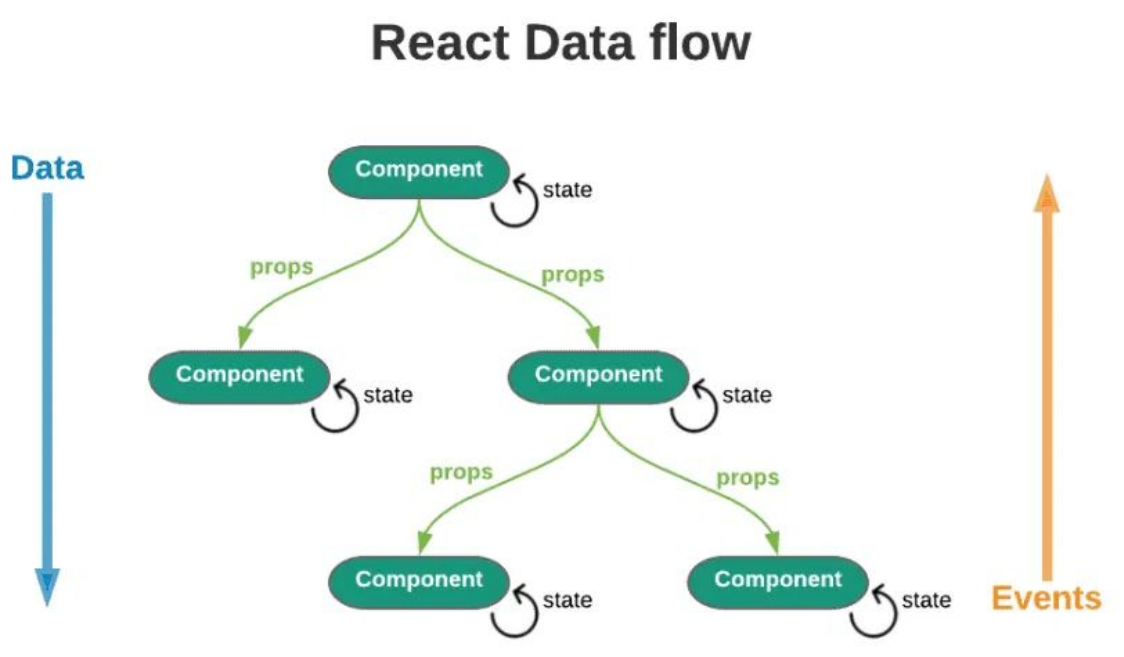
Índice

- INTRODUÇÃO
- PROPS
- MÚLTIPLAS PROPRIEDADES
- PROPS E VARIÁVEIS
- PROPS DE OBJETOS
- PROPS DE ARRAYS
- DESESTRUTURANDO PROPS
- PROPS CHILDREN
- COMUNICAÇÃO E ELEVAÇÃO DE ESTADO
- ESTADO LOCAL (usestate)
- EXERCÍCIOS

Introdução

FLUXO DE DADOS UNIDIRECIONAL

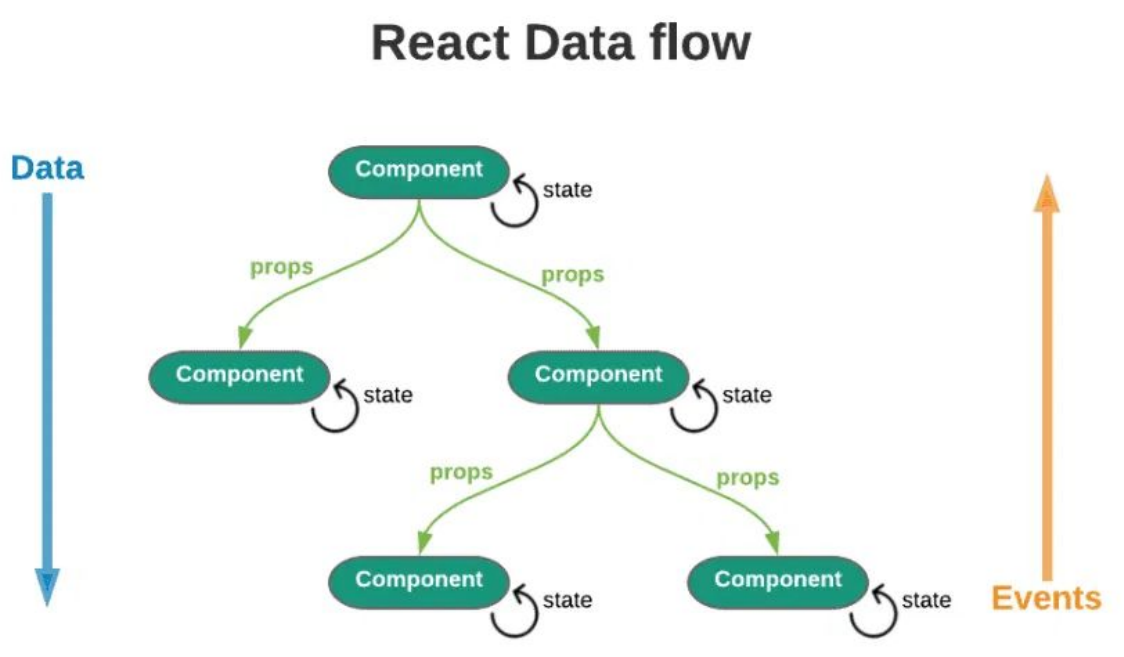
- O React fundamenta-se em um fluxo de dados unidirecional
- As informações viajam de cima para baixo na hierarquia de componentes
- Esta arquitetura transforma componentes em funções previsíveis



Introdução

FLUXO DE DADOS UNIDIRECIONAL

- Elas recebem entradas e retornam elementos de interface
- Isso tornar o caminho dos dados explícito
- O que facilita a depuração e a manutenção de sistemas complexos



Props

CONCEITO

- As Props funcionam como argumentos de uma função
- Permitem a passagem de dados de um componente pai para um filho
- São passadas via atributos HTML
- props vem de *properties*

Props

```
createRoot(document.getElementById('root')).render(

  <Car brand="Ford" />

);
```

```
function Car(props) {

  return (

    <h2>I am a {props.brand}!</h2>

  ); //Usando o atributo brand

}
```

Props

```
createRoot(document.getElementById('root')).render(

  <Car brand="Ford" />

);
```

```
function Car(props) {

  return (

    <h2>I am a {props.brand}!</h2>

  ); //Usando o atributo brand

}
```

Props

```
createRoot(document.getElementById('root')).render(

  <Car brand="Ford" />

);
```

```
function Car(myobj) {

  return (

    <h2>I am a {myobj.brand}!</h2>

  ); // O objeto pode ter qualquer nome, não obrigatoriamente props

}
```

Props

```
createRoot(document.getElementById('root')).render(

  <Car brand="Ford" />

);
```

```
function Car(myobj) {

  return (

    <h2>I am a {myobj.brand}!</h2>

  ); // O objeto pode ter qualquer nome, não obrigatoriamente props

}
```

Múltiplas Propriedades

```
createRoot(document.getElementById('root')).render(

  <Car brand="Ford" model="Mustang" color="red" />

);
```

```
function Car(props) {

  return (

    <h2>I am a {props.color} {props.brand} {props.model}!</h2>

  );

}
```

Props e variáveis

- As propriedades (props) do React podem ser de qualquer tipo de dado, incluindo variáveis, números, strings, objetos, arrays e muito mais
- Strings podem ser enviadas entre aspas, como feito anteriormente, mas números, variáveis e objetos precisam ser enviados entre chaves

```
createRoot(document.getElementById('root')).render(
  <Car year={1969} />
);
```

Props e variáveis

- Variáveis

```
let x = "Ford";



createRoot(document.getElementById('root')).render(

  <Car brand={x} />

);
```

Props e variáveis

- Objetos e arrays

```
let x = [1964, 1965, 1966];

let y = {name: "Ford", model: "Mustang"};



createRoot(document.getElementById('root')).render(

  <Car years={x} carinfo={y} />

);
```

Props de Objetos

- O componente trata os objetos como objetos
- A notação de ponto pode ser usada para acessar as propriedades

```
const carInfo = {

  name: "Ford",

  model: "Mustang",

  color: "red",

  year: 1969

};
```

Props de Objetos

```
function Car(props) {

  return (

    <>

      <h2>My {props.carinfo.name} {props.carinfo.model}!</h2>

      <p>It is {props.carinfo.color} and it is from
{props.carinfo.year}!</p>

    </>

  );

}
```

Props de Arrays

```
function Car(props) {
  return (
    <h2>My car is a {props.carinfo[0]} {props.carinfo[1]}!</h2>
  );
}
const carInfo = ["Ford", "Mustang"];
createRoot(document.getElementById('root')).render(
  <Car carinfo={carInfo} />
);
```

Desestruturando Props

- É possível limitar as propriedades que um componente recebe usando a desestruturação

```
function Car({color}) {
  return (
    <h2>My car is {color}!</h2>
  );
} // O componente sabe que precisa apenas de color
createRoot(document.getElementById('root')).render(
  <Car brand="Ford" model="Mustang" color="red" year={1969} />
);
```

Desestruturando Props

- É possível também desestruturar as propriedades necessárias dentro do componente
- Dessa forma, o componente recebe todas as propriedades
- Mas a desestruturação garante que ele utilize apenas as que precisa

Desestruturando Props

```
function Car(props) {

  const {brand, model} = props;

  return (

    <h2>I love my {brand} {model}!</h2>

  );

}

createRoot(document.getElementById('root')).render(

  <Car brand="Ford" model="Mustang" color="red" year={1969} />

);odel="Mustang" color="red" year={1969} />

);
```

...rest

- Quando não se sabe quantas propriedades serão recebidas, pode-se usar o operador ...rest.
- Significado: especificar as propriedades necessárias e o restante será armazenado em um objeto
- No exemplo a seguir, o componente especifica a cor e a marca, mas o restante é armazenado em um objeto como este:

{ model: "Mustang", year: 1969 }.

...rest

```
function Car({color, brand, ...rest}) {

  return (

    <h2>My {brand} {rest.model} is {color}!</h2>

  );

}



createRoot(document.getElementById('root')).render(

  <Car brand="Ford" model="Mustang" color="red" year={1969} />

);
```

Valores padrão

- Com a desestruturação, é possível definir valores padrão para as propriedades
- Se uma propriedade não tiver valor, o valor padrão será usado

```
function Car({color = "blue", brand}) {
  return (
    <h2>My {color} {brand}!</h2>
  );
}

createRoot(document.getElementById('root')
).render(
  <Car brand="Ford" />
);
```

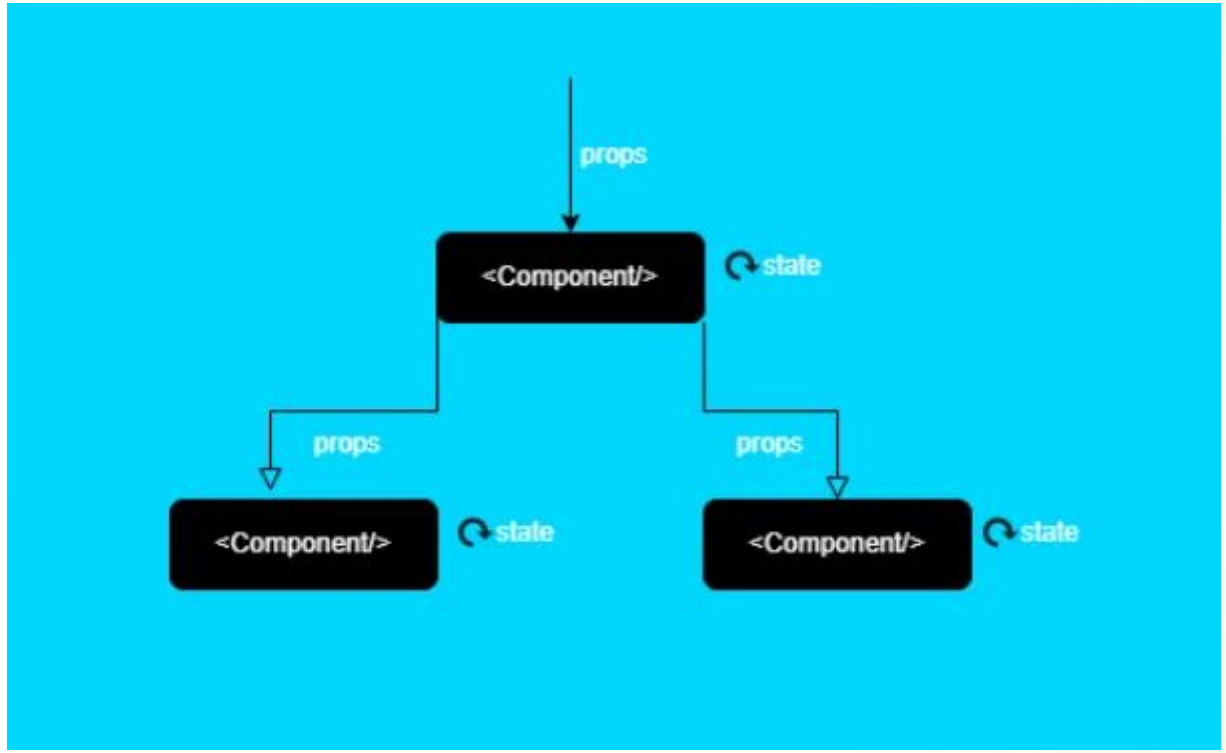
Props Children

- Em React, você pode enviar o conteúdo entre as tags de abertura e fechamento de um componente para outro componente
- Esse conteúdo pode ser acessado no outro componente usando a propriedade `props.children`
- No exemplo a seguir, a partir do componente Pai, o conteúdo é enviado entre as tags de abertura e fechamento dos componentes Filho e Filha
- Ver exemplo no W3Schools

Comunicação e Elevação de Estado

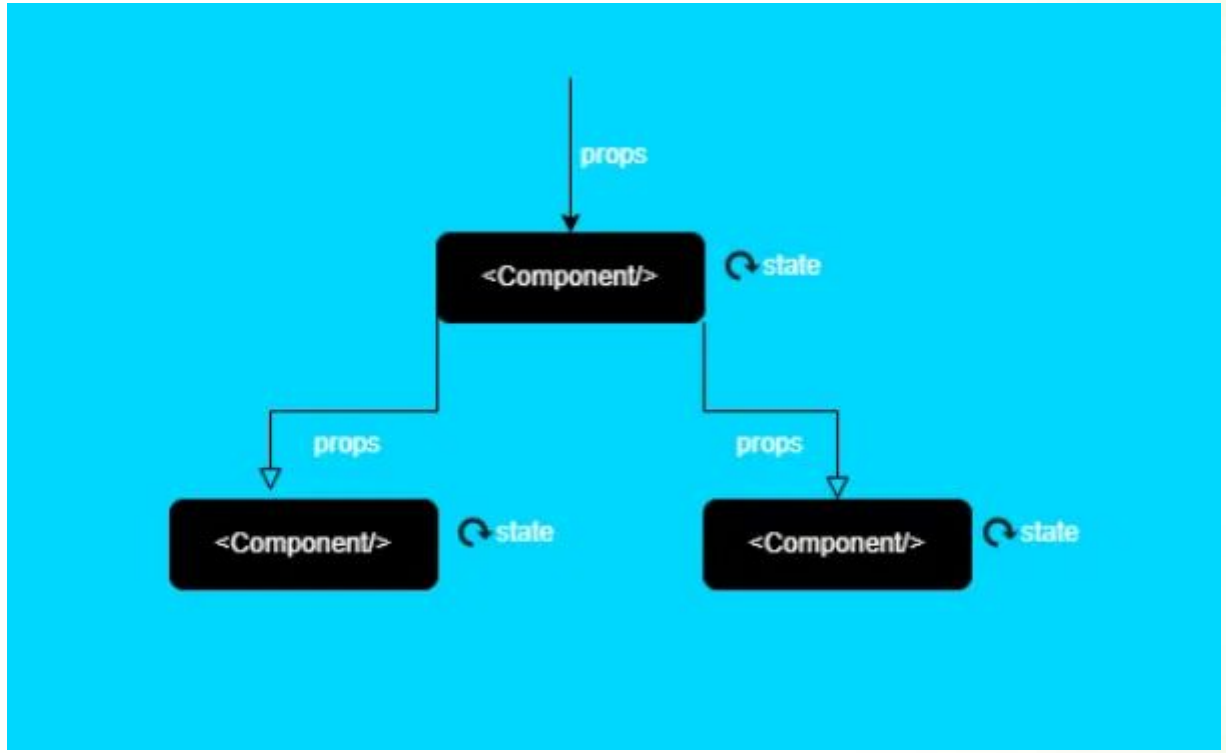
- A comunicação via props permite que componentes filhos notifiquem seus pais sobre eventos através de funções de callback

- O padrão de Elevação de Estado (Lifting State Up) consiste em mover o estado para o ancestral comum mais próximo



Comunicação e Elevação de Estado

- Isso garante que múltiplos componentes permaneçam sincronizados, compartilhando a mesma fonte de verdade



Hooks React

- Os hooks permitem que funções acessem o estado e outros recursos do React sem usar classes
- Eles fornecem uma API mais direta para conceitos do React como props, state, context, refs e lifecycle
- Hooks são funções que permitem "conectar-se" ao estado e aos recursos do ciclo de vida do React a partir de componentes funcionais

Estado Local (useState)

- Enquanto as props são externas, o Estado representa a memória interna de um componente.
- O hook useState permite que o componente "lembre" de informações entre renderizações.
- Sempre que o estado é atualizado, o React agenda uma nova renderização para atualizar a interface de forma eficiente.

Estado Local (useState)

- O Hook useState do React permite rastrear o estado em um componente funcional
- Estado geralmente se refere a dados ou propriedades que precisam ser rastreados em uma aplicação
- Importar useState: Para usar o Hook useState, primeiro ele é importado no topo do componente:

```
import { useState } from "react";
```

Estado Local (useState)

- O estado é inicializado chamando "useState" no componente de função.
- "useState" aceita um estado inicial e retorna dois valores:
  - O estado atual
  - Uma função que atualiza o estado

```
import { useState } from "react";

function FavoriteColor() {

  const [color, setColor] = useState("red");

}
```

Estado Local (useState)

- O primeiro valor, "color", é o estado atual
- O segundo valor, "setColor", é a função usada para atualizar o estado
- Esses nomes são variáveis que podem ter qualquer nome
- Por fim, define-se o estado inicial como "red": "useState("red")"

Estado Local (useState)

LENDO O ESTADO

```
import { useState } from 'react';

import { createRoot } from 'react-dom/client';

function FavoriteColor() {

  const [color, setColor] = useState("red");



  return <h1>My favorite color is {color}!</h1>

}

createRoot(document.getElementById('root')).render(

  <FavoriteColor />

);
```

Estado Local (useState)

ATUALIZANDO O ESTADO

- Para atualizar o estado, utiliza-se a função de atualização de estado
- Nunca deve-se atualizar diretamente o estado. Ex: color = "blue"

```
<button
  type="button"
  onClick={() => setColor("blue")}
>Blue</button>
```

Exercícios

EXERCÍCIO 1 - PROPS

- Crie um componente UserCard que exiba foto, nome, cargo e um booleano isOnline
- Se isOnline for true, mostre uma bolinha verde; se false, cinza

Exercícios

EXERCÍCIO 2 - USE STATE

- Crie um componente que simula uma lâmpada
- Ele deve ter um botão "Ligar/Desligar"
- O fundo da div deve mudar de cor (preto/amarelo) e o texto do botão deve alternar

Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- Entregas da aula de hoje: Fundamentos e Virtual DOM
  - Criação do projeto React
  - Criação dos componentes iniciais
  - Uso correto de JSX
  - Renderização estática dos dados
- Funcionalidade: Estrutura estática da tela (HTML/CSS transformado em JSX)
- Entrega: Componentes (App, Task)

Exercício: Gerenciador de Tarefas

- Entregas da aula de hoje: Props e UseState
  - Transformar a lista estática em um Array no useState
  - Passar dados do Pai (App) para o Filho (TaskItem) via Props
  - Criar interatividade: Botão de "Concluir" (risca o texto) e "Remover" (tira da lista)
  - Input controlado para adicionar novas tarefas
- Pode-se digitar, dar Enter e a tarefa aparece
- Quando clica, a tarefa some. (Ao dar F5, tudo some).